

Real Anime Reviews - Project Handoff

Updated handoff for continuing work in a new GPT session (Dec 2025)

1. What this project is

Real Anime Reviews is a static (no framework) anime review website built with vanilla HTML/CSS/JS. It has a main browsing page (index.html) and an account page (account.html). Firebase provides Auth, Firestore, Storage (avatars), and Hosting.

2. Current production setup (what is live)

- Firebase Hosting project: real-anime-reviews
- Default Hosting URL (always works): <https://real-anime-reviews.web.app>
- Custom domain: realanimereviews.com (bought on Namecheap, DNS points to Firebase)
- Deployment method: Firebase CLI from the project folder (firebase deploy --only hosting)
- This is a static hosting deployment (Firebase Hosting), not Firebase App Hosting.

3. Repo / folder structure (what files do what)

These are the core files you will be editing most often:

- index.html - main landing page, anime cards, anime modal, community tab UI container, auth modal container.
- account.html - user profile + My Activity page + notifications UI.
- style.css - all UI styling (cards, modal, community tab, notification panel, activity list).
- animeData.js - the anime database (global const animeData = [...]).
- firebase.js - Firebase initialization + exports used by script.js/account.js.
- script.js - main site logic (rendering, modal, comments, community reviews, voting, notifications).
- account.js - account/profile page logic (profile, avatar upload, activity feed, notifications).
- firebase.json + .firebaserc - hosting configuration + the Firebase project id mapping.
- 404.html - hosting fallback page (keep it).

4. Data sources

Anime entries are stored in animeData.js as a global array. An Excel master table also exists (Anime_Master_Table.xlsx) as the human-editing source of truth, but the site reads animeData.js.

5. Firebase products in use

- Authentication - users can sign up / log in. UI is on the site pages (not a separate app).
- Cloud Firestore - all user-generated content and user state (favorites/watchlist/votes/comments/reviews/threads/notifications).
- Cloud Storage - user avatar uploads.
- Firebase Hosting - production hosting + preview channels for safe testing.

6. Firestore schema (collections and what they store)

Important paths (these must match rules + code):

- /users/{uid} - user profile doc (displayName, photoURL, etc.)
- /users/{uid}/favorites/{animelId} - per-user favorites (doc id = animelId/slug)
- /users/{uid}/watchlist/{animelId} - per-user watchlist (doc id = animelId/slug)
- /users/{uid}/notifications/{notifId} - notification inbox for that user (vote notifications)
- /comments/{animelId}/items/{cid} - public comments under an anime
- /comments/{animelId}/items/{cid}/votes/{voterUid} - one vote per user per comment (value: 1 or -1)
- /reviews/{animelId}/items/{reviewAuthorUid} - community reviews (one per user per anime)
- /reviews/{animelId}/items/{reviewAuthorUid}/votes/{voterUid} - votes on that community review
- /reviews/{animelId}/items/{reviewAuthorUid}/threads/{tid} - discussion comments under a community review
- /reviews/{animelId}/items/{reviewAuthorUid}/threads/{tid}/votes/{voterUid} - votes on discussion comments
- /official/{animelId} - aggregate vote doc for the site owner's official rating for that anime
- /official/{animelId}/votes/{voterUid} - each user vote on the official rating (value: 1 or -1)

7. Security rules (known-good baseline)

Below is a clean, complete Firestore rules file that matches the current feature set. If rules ever get messy again (duplicate blocks, missing braces), replace your entire rules with this file as a baseline and then re-test like/dislike + notifications + official votes.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function isSignedIn() { return request.auth != null; }
    function isOwner(uid) { return isSignedIn() && request.auth.uid == uid; }

    // =====
    // USER PROFILES
```

```

// =====
match /users/{uid} {
  allow get: if true; // public profile read (tighten later)
  allow create, update, delete: if isOwner(uid);

  match /favorites/{animeId} {
    allow get, list: if isOwner(uid);
    allow create, update, delete: if isOwner(uid);
  }

  match /watchlist/{animeId} {
    allow get, list: if isOwner(uid);
    allow create, update, delete: if isOwner(uid);
  }

  // NOTIFICATIONS (vote notifications delivered to the owner)
  match /notifications/{notifId} {
    allow get, list: if isOwner(uid);

    // any signed-in user can CREATE a notification to someone else
    // but it must be "from themselves" and "to this uid"
    allow create: if isSignedIn()
      && request.resource.data.toUid == uid
      && request.resource.data.fromUid == request.auth.uid
      && request.resource.data.type is string
      && (request.resource.data.type == "comment_vote" || request.resource.data.type ==
"review_vote")
      && request.resource.data.value is int
      && (request.resource.data.value == 1 || request.resource.data.value == -1)
      && request.resource.data.animeId is string
      && request.resource.data.animeTitle is string
      && request.resource.data.targetId is string
      && request.resource.data.fromDisplayName is string
      && (!('fromPhotoURL' in request.resource.data) ||
request.resource.data.fromPhotoURL is string || request.resource.data.fromPhotoURL ==
null)
      && ('createdAt' in request.resource.data);

    // owner can manage their own notifications
    allow update, delete: if isOwner(uid);
  }
}

```

```

// =====
// COLLECTION GROUP READS (activity feed)
// Note: only allow reading your own docs by uid.
// =====
match /{parentPath=**}/items/{docId} {
  allow get, list: if isSignedIn() && resource.data.uid == request.auth.uid;
}

match /{parentPath=**}/threads/{docId} {
  allow get, list: if isSignedIn() && resource.data.uid == request.auth.uid;
}

// =====
// COMMENTS
// =====
match /comments/{animeId}/items/{cid} {
  allow get, list: if true;

  allow create: if isSignedIn()
    && request.resource.data.uid == request.auth.uid
    && request.resource.data.text is string
    && request.resource.data.displayName is string
    && (!('photoURL' in request.resource.data) || request.resource.data.photoURL is string
|| request.resource.data.photoURL == null)
    && ('createdAt' in request.resource.data)
    && request.resource.data.likesCount is int
    && request.resource.data.dislikesCount is int;

  allow delete: if isSignedIn() && resource.data.uid == request.auth.uid;

  // Author edits OR vote count updates
  allow update: if (
    isSignedIn()
    && resource.data.uid == request.auth.uid
    &&
request.resource.data.diff(resource.data).changedKeys().hasOnly(['text','editedAt','photoURL',
'displayName'])
  )
  || (
    isSignedIn()
    &&
request.resource.data.diff(resource.data).changedKeys().hasOnly(['likesCount','dislikesCount'
])
  )
}

```

```

    && request.resource.data.likesCount is int
    && request.resource.data.dislikesCount is int
    && request.resource.data.likesCount >= 0
    && request.resource.data.dislikesCount >= 0
  );

match /votes/{voterUid} {
  allow get, list: if true;

  allow create, update: if isOwner(voterUid)
    && request.resource.data.value is int
    && (request.resource.data.value == 1 || request.resource.data.value == -1)
    && ('updatedAt' in request.resource.data);

  allow delete: if isOwner(voterUid);
}
}

// =====
// COMMUNITY REVIEWS
// =====
match /reviews/{animeId}/items/{reviewUid} {
  allow get, list: if true;

  allow create: if isOwner(reviewUid)
    && request.resource.data.uid == request.auth.uid
    && request.resource.data.displayName is string
    && (!('photoURL' in request.resource.data) || request.resource.data.photoURL is string
|| request.resource.data.photoURL == null)
    && request.resource.data.title is string
    && request.resource.data.body is string
    && request.resource.data.rating is number
    && request.resource.data.rating >= 1 && request.resource.data.rating <= 10
    && ('createdAt' in request.resource.data)
    && ('updatedAt' in request.resource.data)
    && request.resource.data.likesCount is int
    && request.resource.data.dislikesCount is int;

  // Author edits OR vote count updates
  allow update: if (
    isOwner(reviewUid)
    && request.resource.data.uid == request.auth.uid
    &&

```

```

request.resource.data.diff(resource.data).changedKeys().hasOnly(['title','body','rating','update
dAt','displayName','photoURL'])
)
|| (
  isSignedIn()
  &&
request.resource.data.diff(resource.data).changedKeys().hasOnly(['likesCount','dislikesCount'
])
  && request.resource.data.likesCount is int
  && request.resource.data.dislikesCount is int
  && request.resource.data.likesCount >= 0
  && request.resource.data.dislikesCount >= 0
);

allow delete: if isOwner(reviewUid);

match /votes/{voterUid} {
  allow get, list: if true;

  allow create, update: if isOwner(voterUid)
    && request.resource.data.value is int
    && (request.resource.data.value == 1 || request.resource.data.value == -1)
    && ('updatedAt' in request.resource.data);

  allow delete: if isOwner(voterUid);
}

// REVIEW DISCUSSION THREADS (comments under a community review)
match /threads/{tid} {
  allow get, list: if true;

  allow create: if isSignedIn()
    && request.resource.data.uid == request.auth.uid
    && request.resource.data.text is string
    && request.resource.data.displayName is string
    && (!('photoURL' in request.resource.data) || request.resource.data.photoURL is string
|| request.resource.data.photoURL == null)
    && ('createdAt' in request.resource.data)
    && request.resource.data.likesCount is int
    && request.resource.data.dislikesCount is int;

  allow delete: if isSignedIn() && resource.data.uid == request.auth.uid;

```

```

// Author edits OR vote count updates
allow update: if (
  isSignedIn()
  && resource.data.uid == request.auth.uid
  &&
request.resource.data.diff(resource.data).changedKeys().hasOnly(['text','editedAt','photoURL',
'displayName'])
)
|| (
  isSignedIn()
  &&
request.resource.data.diff(resource.data).changedKeys().hasOnly(['likesCount','dislikesCount'
])
  && request.resource.data.likesCount is int
  && request.resource.data.dislikesCount is int
  && request.resource.data.likesCount >= 0
  && request.resource.data.dislikesCount >= 0
);

match /votes/{voterUid} {
  allow get, list: if true;

  allow create, update: if isOwner(voterUid)
    && request.resource.data.value is int
    && (request.resource.data.value == 1 || request.resource.data.value == -1)
    && ('updatedAt' in request.resource.data);

  allow delete: if isOwner(voterUid);
}
}
}

// =====
// OFFICIAL (Blake) REVIEW VOTES (per anime)
// Path: official/{animeId}
// Vote doc: official/{animeId}/votes/{voterUid}
// =====
match /official/{animeId} {
  allow get, list: if true;

  // aggregate doc is created/updated by signed-in voters via transaction
  allow create: if isSignedIn()
    && request.resource.data.animeId is string

```

```

    && request.resource.data.likesCount is int
    && request.resource.data.dislikesCount is int
    && request.resource.data.likesCount >= 0
    && request.resource.data.dislikesCount >= 0
    && ('updatedAt' in request.resource.data);

    allow update: if isSignedIn()
    &&
    request.resource.data.diff(resource.data).changedKeys().hasOnly(['animeId','likesCount','dislikesCount','updatedAt'])
    && request.resource.data.likesCount is int
    && request.resource.data.dislikesCount is int
    && request.resource.data.likesCount >= 0
    && request.resource.data.dislikesCount >= 0;

    allow delete: if false;

    match /votes/{voterUid} {
      allow get, list: if true;

      allow create, update: if isOwner(voterUid)
      && request.resource.data.value is int
      && (request.resource.data.value == 1 || request.resource.data.value == -1)
      && ('updatedAt' in request.resource.data);

      allow delete: if isOwner(voterUid);
    }
  }
}

```

8. Firestore indexes you needed (so queries stop failing)

To power the My Activity feed efficiently, the project uses collection-group queries on items and threads. Firestore will ask you to create indexes the first time you run certain queries. If you reset the project or create a new Firebase project, you will need to recreate these indexes.

- Collection group: items - index fields include uid (Ascending) + createdAt (Descending)
- Collection group: threads - index fields include uid (Ascending) + createdAt (Descending)

9. Notification system behavior (limits + scrolling)

Notifications are stored under `/users/{uid}/notifications`. The UI panel should stay a fixed height and scroll instead of growing forever. The code also deletes older notifications (keep a small fixed number, e.g., 10) after a new notification arrives, so the database does not grow without bound.

10. Deployment workflow (how to safely test updates)

There are 3 stages you should use:

- Local dev: run with VS Code Live Server on `http://127.0.0.1:5500` (fast iteration).
- Preview channel (safe share): deploy to a temporary Firebase Hosting preview URL for testing.
- Production: deploy to the live site (`realanimereviews.com + web.app`).

Preview channel commands (run in the project folder):

```
firebase hosting:channel:deploy staging
# CLI prints a preview URL like https://staging--real-anime-reviews.web.app

firebase hosting:channel:list
firebase hosting:channel:delete staging
```

Production deploy (after preview looks good):

```
firebase deploy --only hosting
```

11. Domain + DNS (what happened and why it worked)

A domain registrar (Namecheap) controls your domain name. DNS records are the public mapping that tells the internet where your domain should point. Firebase Hosting cannot control Namecheap DNS automatically, so you had to copy Firebase's records into Namecheap.

The working setup for `realanimereviews.com` is:

- A record (Host `@`) -> `199.36.158.100` (Firebase Hosting IP for apex domain)
- CNAME record (Host `www`) -> `ghs.googlehosted.com` (points `www.realanimereviews.com` to Firebase)
- TXT record (Host `@`) -> `hosting-site=real-anime-reviews` (domain ownership verification token shown by Firebase)

After those records existed, Firebase verified domain ownership, then provisioned SSL automatically. DNS changes can take minutes to hours to propagate.

12. How to make Discord / social previews look legit

Discord uses Open Graph tags (og:title, og:description, og:image). If your page does not provide them, Discord shows a minimal or sketchy preview. Add OG meta tags to index.html <head> and use an absolute image URL.

```
<!-- Put inside <head> of index.html -->
<meta property="og:title" content="Real Anime Reviews" />
<meta property="og:description" content="Real ratings from a normal guy. Browse anime
reviews, community ratings, and discussions." />
<meta property="og:type" content="website" />
<meta property="og:url" content="https://realanimereviews.com/" />
<meta property="og:image" content="https://realanimereviews.com/assets/og-preview.jpg" />
<meta name="twitter:card" content="summary_large_image" />
```

Discord caches previews. If you change OG tags, use a new URL once (add ?v=2) to force a refresh.

13. How we work together (the surgical copy-paste rules)

This project stays stable by doing small, controlled edits:

- Never rewrite entire files unless absolutely necessary.
- Always locate an anchor comment or a unique line, then paste code exactly there.
- Edits are delivered as: file name + EXACT find line + paste below / replace this block.
- After changes: reload locally first, then preview channel, then production deploy.
- Keep a backup ZIP of the last known-good version before any risky change.

14. Quick checklist for a new GPT picking this up

1. Ask the user for the current ZIP (or the current folder contents) so you are never patching an outdated copy.
2. Confirm the Firebase project id is still real-anime-reviews and that hosting deploy works from the folder.
3. Confirm Firestore rules compile and voting + notifications work.
4. Use preview channels for any non-trivial UI/Firestore change.
5. When making a change, give the user exact instructions: which file, which line to search for, and the code to paste.

15. How to instruct Blake (communication style for another GPT)

Blake prefers extremely explicit, step-by-step instructions and copy-paste blocks. When giving changes:

- Start with the file name(s).
- Tell him exactly what to search for (Ctrl+F) so he lands on the correct spot.
- Say “replace everything from LINE A to LINE B” OR “paste this block directly below this exact line”.
- Provide one change at a time; wait for him to confirm it works before stacking more changes.
- Avoid vague guidance like “just configure DNS” — instead: “click Advanced DNS → Add New Record → choose TXT → Host @ → Value ... → click the green check”.
- After each change: (1) test locally, (2) deploy preview channel, (3) deploy production.

Useful pattern for code patches:

FILE: style.css

FIND: /* Community review titles */

REPLACE THAT BLOCK WITH:

...